



BURSA ULUDAĞ ÜNİVERSİTESİ

2025-2026 ÖĞRETİM YILI BAHAR DÖNEMİ

VERİ YAPILARI PROJE RAPORU

Proje Adı : PCB BAĞI OPTİMİZASYONU

Bölüm: Bilgisayar Mühendisliği

Ders Yürütücüsü: Murtaza CİCİOĞLU / Fatih KAYA

Hazırlayan Ad-Soyad-Numara-Sınıf:

Elif ÖZDEMİR - 032490086 - 2.Sınıf

Cağfer Tuğra ÇETİN - 032490088 - 2.Sınıf

Niyazi Han AKBULUT - 032490091 - 2. Sınıf

Azra TAŞHAN - 032490092 - 2. Sınıf

Evin YILMAZ - 032490093 - 2.Sınıf

1.Proje Amacı

PCB üzerindeki bağlantı noktalarını bir graf yapısı ile modelleyerek en uygun bağlantı ağını oluşturmaktır. PCB üzerindeki noktalar grafın düğümleri, bu noktalar arasındaki bağlantılar ise grafın kenarları olarak temsil edilmiştir. Her kenarın bir ağırlığı bulunur ve bu ağırlık bağlantı maliyetini, uzaklığı veya işlem yükünü ifade eder.

Proje kapsamında graf veri yapısı üzerinde temel işlemler gerçekleştirilmiş, grafın bağlılık durumu kontrol edilmiş ve minimum maliyetli bağlantı ağı elde etmek için Kruskal algoritması uygulanmıştır. Ayrıca kullanıcı arayüzü ile C tabanlı backend arasında JSON dosyaları üzerinden veri alışverişi sağlanmıştır.

2.Kullanılan Teknolojiler

Projede aşağıdaki teknolojiler kullanılmıştır:

- C programlama dili
- CMake
- Docker
- Docker Compose
- Python Dash / Flask tabanlı kullanıcı arayüzü
- JSON veri formatı
- GitHub versiyon kontrol sistemi

C dili, graf yapısının ve algoritmaların verimli biçimde uygulanması için tercih edilmiştir. Kullanıcı arayüzü ise graf girişinin daha kolay yapılabilmesi ve sonuçların görsel olarak gösterilebilmesi amacıyla geliştirilmiştir.

3. Sistem Mimarisi

Proje frontend ve backend olmak üzere iki ana bölümden oluşmaktadır.

Frontend, kullanıcının graf verisini girmesini ve sonuçları görmesini sağlar. Backend ise C dili ile yazılmıştır ve graf üzerinde algoritmik işlemleri gerçekleştirir. Frontend ile backend arasındaki iletişim doğrudan fonksiyon çağırısı ile değil, ortak bir veri klasörü üzerinden sağlanmaktadır.

Sistem genel olarak şu şekilde çalışmaktadır:

1. Kullanıcı arayüz üzerinden graf verisini girer.
2. Frontend bu veriyi **input_graph.json** dosyasına yazar.
3. Frontend, backend'in hesaplama yapması için **calculate.flag** dosyasını oluşturur.
4. Backend bu flag dosyasını sürekli kontrol eder.

5. Flag dosyası bulunduğunda backend JSON dosyasını okur.
6. Graf yapısı oluşturulur.
7. Grafin bağlı olup olmadığı kontrol edilir.
8. Kruskal algoritması çalıştırılır.
9. Minimum spanning tree sonucu **output_mst.json** dosyasına yazılır.
10. Frontend bu sonucu okuyarak kullanıcıya gösterir.

Bu yapı sayesinde frontend ve backend birbirinden bağımsız çalışabilir hale getirilmiştir.

4. Docker ile Çalıştırma

Docker Compose kullanılarak tek komutla çalıştırılabilir hale getirilmiştir. Böylece kullanıcıların C derleyicisi, CMake, Python bağımlılıkları veya ek kütüphaneleri manuel olarak kurmasına gerek kalmadan proje container ortamında çalıştırılabilir.

Proje ana dizininde aşağıdaki komut çalıştırılarak sistem başlatılır:

```
docker compose up --build
```

Bu komut ile backend ve frontend servisleri birlikte oluşturulur ve çalıştırılır.

Backend servisi C kodlarını container içerisinde derler ve çalıştırır. Frontend servisi ise kullanıcı arayüzünü başlatır. Arayüz aşağıdaki adresten erişilebilir:

<http://localhost:8050>

Docker Compose yapılandırmasında frontend ve backend servisleri tanımlanmıştır. Backend servisi yalnızca ortak veri klasörünü kullanarak frontend ile haberleşmektedir. Bu sayede container içinde derlenen çalıştırılabilir dosyanın üzerine host klasörü bind edilerek bozulması engellenmiştir.

5. Veri Yapıları

Projede temel veri yapısı olarak graf kullanılmıştır. Graf, PCB bağlantı ağını temsil etmek için kullanılır.

5.1 Graph Veri Yapısı

Graph veri yapısı, düğümler ve kenarlar arasındaki bağlantıları saklamak için kullanılmıştır. PCB üzerindeki her bağlantı noktası bir düğüm olarak, iki nokta arasındaki bağlantı ise kenar olarak temsil edilmiştir.

Graf yapısında temel olarak şu bilgiler tutulur:

- Düğüm sayısı
- Kenar sayısı

- Komşuluk matrisi veya kenar listesi
- Kenarların kaynak düğümü
- Kenarların hedef düğümü
- Kenar ağırlıkları

Bu yapı sayesinde graf üzerinde BFS, DFS, bağlılık kontrolü ve Kruskal algoritması gibi işlemler yapılabilir.

5.2 Edge Veri Yapısı

Edge yapısı, graf üzerindeki iki düğüm arasındaki bağlantıyı temsil eder. Her kenar kaynak düğüm, hedef düğüm ve ağırlık bilgisinden oluşur.

Bu yapı özellikle Kruskal algoritmasında kullanılmıştır. Çünkü Kruskal algoritması, kenarları ağırlıklarına göre sıralayarak minimum spanning tree oluşturur.

5.3 Queue Veri Yapısı

Queue yapısı BFS algoritmasında kullanılmıştır. BFS algoritması düğümleri seviye seviye gezdiği için FIFO mantığına ihtiyaç duyar.

Queue işlemleri:

- enqueue
- dequeue
- isEmpty

5.4 Stack Veri Yapısı

Stack yapısı DFS algoritmasında kullanılabilecek temel veri yapılarından biridir. DFS, derinlemesine ilerlediği için LIFO mantığına uygun çalışır.

Stack işlemleri:

- push
- pop
- isEmpty

5.5 Union-Find Veri Yapısı

Union-Find veri yapısı Kruskal algoritmasında döngü oluşumunu engellemek için kullanılmıştır. Kruskal algoritması kenarları küçükten büyüğe seçerken, seçilen kenarın graf içinde cycle oluşturup oluşturmadığını kontrol etmelidir.

Union-Find veri yapısı şu işlemleri içerir:

- find
- union
- parent takibi
- küme birleştirme

Bu yapı sayesinde iki düğümün aynı bağlı bileşende olup olmadığı hızlı şekilde kontrol edilir.

6. Kullanılan Algoritmalar

6.1 BFS Algoritması

BFS, graf üzerinde genişlik öncelikli arama yapmak için kullanılır. Başlangıç düğümünden itibaren önce komşu düğümler ziyaret edilir, sonra bir sonraki seviyeye geçilir.

Bu algoritma özellikle grafın gezilmesi ve bağlantılık kontrolü için uygundur.

BFS algoritmasının zaman karmaşıklığı:

$$O(V + E)$$

Burada V düğüm sayısını, E ise kenar sayısını ifade eder.

6.2 DFS Algoritması

DFS, graf üzerinde derinlik öncelikli arama yapmak için kullanılır. Başlangıç düğümünden itibaren mümkün olduğunca derine gidilir, daha sonra geri dönülerek diğer yollar incelenir.

DFS algoritmasının zaman karmaşıklığı:

$$O(V + E)$$

DFS de bağlantılık kontrolü ve graf gezme işlemlerinde kullanılabilir.

6.3 Connectedness Kontrolü

Connectedness kontrolü, grafın bağlı olup olmadığını anlamak için yapılır. Eğer bir graf bağlı değilse, tüm düğümleri kapsayan bir minimum spanning tree oluşturulamaz. Bu yüzden Kruskal algoritması çalıştırılmadan önce grafın bağlantılık durumu kontrol edilir.

Graf bağlı ise tüm düğümlere başlangıç düğümünden ulaşılabilir. Eğer ulaşılamayan düğüm varsa graf bağlı değildir.

Zaman karmaşıklığı:

$$O(V + E)$$

6.4 Kruskal Algoritması

Kruskal algoritması, ağırlıklı ve bağlı bir grafta minimum spanning tree oluşturmak için kullanılmıştır. Algoritma, tüm kenarları ağırlıklarına göre küçükten büyüğe sıralar. Daha sonra sırayla kenarları seçer. Eğer seçilen kenar cycle oluşturmuyorsa MST'ye eklenir.

Kruskal algoritmasının temel adımları:

1. Tüm kenarlar ağırlıklarına göre sıralanır.

2. Her düğüm başlangıçta ayrı bir küme kabul edilir.
3. En küçük ağırlıklı kenardan başlanır.
4. Kenarın iki ucu farklı kümelerdeyse kenar MST'ye eklenir.
5. Bu iki küme union işlemi ile birleştirilir.
6. MST içinde V-1 adet kenar olana kadar işlem devam eder.

Kruskal algoritmasının zaman karmaşıklığı:

$O(E \log E)$

Bu karmaşıklığın temel sebebi kenarların sıralanmasıdır.

7. Zaman Karmaşıklığı Analizi

Projede kullanılan temel veri yapıları ve algoritmalar için zaman karmaşıklıkları aşağıdaki gibidir:

İşlem / Algoritma	Zaman Karmaşıklığı	Açıklama
Graf oluşturma	$O(V + E)$	Düğüm ve kenarlar belleğe alınır.
Kenar ekleme	$O(1)$	Matris veya liste yapısına göre sabit zamanda ekleme yapılabilir.
BFS	$O(V + E)$	Her düğüm ve kenar en fazla bir kez ziyaret edilir.
DFS	$O(V + E)$	Her düğüm ve kenar en fazla bir kez ziyaret edilir.
Connectedness kontrolü	$O(V + E)$	BFS veya DFS ile tüm düğümlere ulaşıp ulaşılmadığı kontrol edilir.
Kenar sıralama	$O(E \log E)$	Kruskal öncesinde kenarlar ağırlığa göre sıralanır.

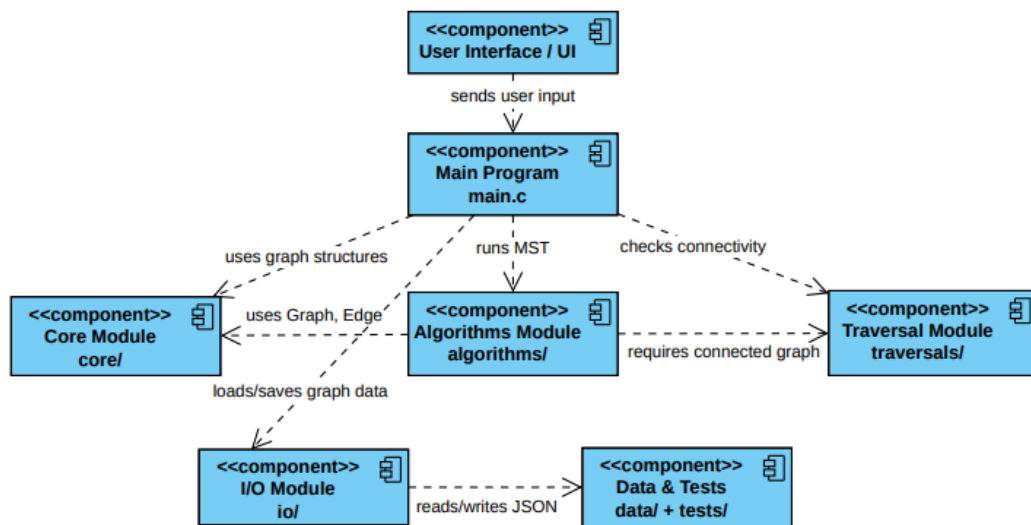
Union-Find find işlemi	Yaklaşık $O(1)$	Path compression kullanıldığında çok verimlidir.
Union-Find union işlemi	Yaklaşık $O(1)$	Kümeler hızlı şekilde birleştirilir.
Kruskal algoritması	$O(E \log E)$	En baskın işlem kenar sıralamasıdır.
JSON okuma/yazma	$O(n)$	Dosya içeriğinin boyutuna bağlıdır.

Genel olarak projenin en maliyetli kısmı Kruskal algoritmasındaki kenar sıralama işlemidir. Bu nedenle genel zaman karmaşıklığı çoğu durumda $O(E \log E)$ olarak değerlendirilebilir.

8 UML Diyagramları

Sistemin yapısını daha anlaşılır hale getirmek için UML diyagramları hazırlanmıştır. UML diyagramları, proje içerisindeki modüllerin görevlerini ve birbirleriyle ilişkilerini görsel olarak göstermektedir.

8.1 Component Diyagramı

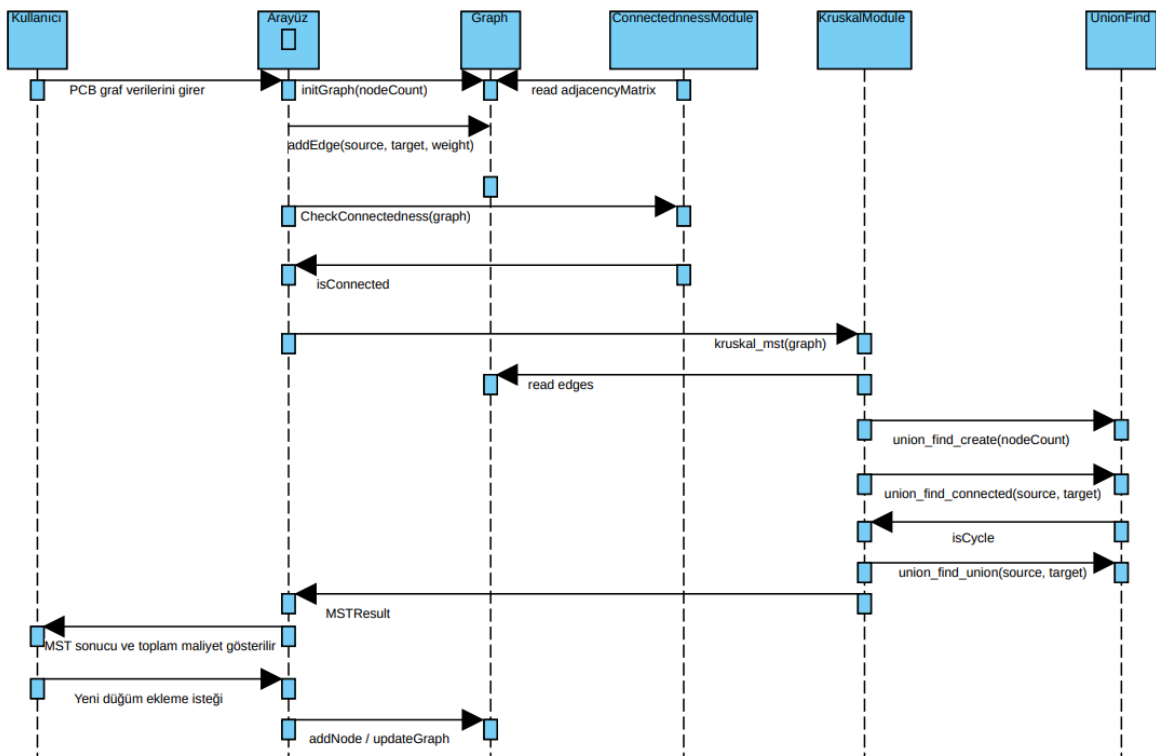


Projenin ana bileşenlerini ve bu bileşenler arasındaki ilişkileri göstermektedir. Kullanıcı arayüzü, kullanıcıdan PCB graf verilerini alır ve bu verileri ana programa iletir. Ana program, graf işlemleri için core modülünü, minimum spanning tree hesaplaması için algorithms modülünü, bağlılık kontrolü için ise traversals modülünü kullanır.

Core modülü graf veri yapısını ve temel graf işlemlerini içerir. algorithms modülü Kruskal algoritmasını çalıştırır. traversals modülü BFS/DFS gibi gezme algoritmalarıyla grafın bağlı olup olmadığını kontrol eder. io modülü ise JSON dosyalarının okunması ve yazılması işlemlerinden sorumludur. data ve tests klasörleri örnek veriler ve test dosyaları için kullanılır.

Bu yapı sayesinde proje modüler bir şekilde tasarlanmıştır. Her bileşen kendi görevinden sorumludur ve sistemin bakımı daha kolay hale gelmiştir.

8.2 Sequence Diagram

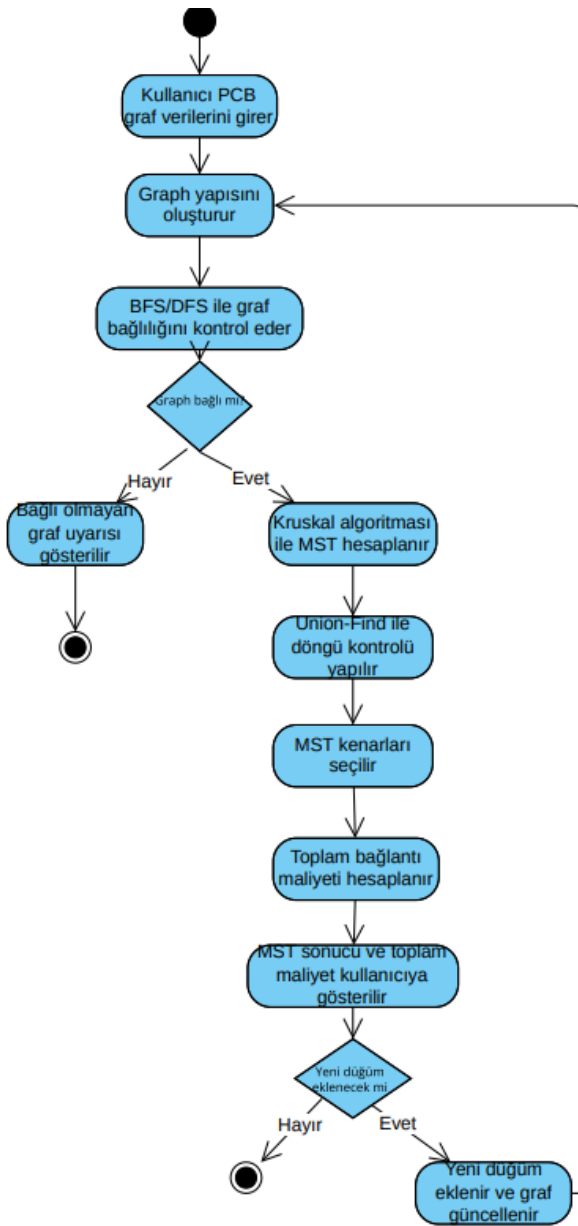


Kullanıcının arayüz üzerinden graf verilerini girmesinden MST sonucunun gösterilmesine kadar geçen süreci adım adım göstermektedir. İlk olarak kullanıcı PCB graf verilerini arayüze girer. Arayüz, graf yapısını oluşturmak için initGraph fonksiyonunu çağırır ve ardından girilen kenarlar için addEdge işlemleri yapılır.

Graf oluşturulduktan sonra bağıllık kontrolü yapılır. ConnectednessModule, grafın tüm düğümlerinin birbirine bağlı olup olmadığını kontrol eder. Eğer graf bağlıysa KruskalModule çalıştırılır. Kruskal algoritması, graf üzerindeki kenarları değerlendirir ve döngü oluşmasını engellemek için UnionFind yapısını kullanır.

Algoritma tamamlandığında MSTResult sonucu arayüze döndürülür. Arayüz, minimum spanning tree kenarlarını ve toplam bağlantı maliyetini kullanıcıya gösterir. Kullanıcı daha sonra yeni düğüm veya bağlantı eklemek isterse graf güncellenerek süreç tekrar devam edebilir.

8.3 Aktivite Diyagramı



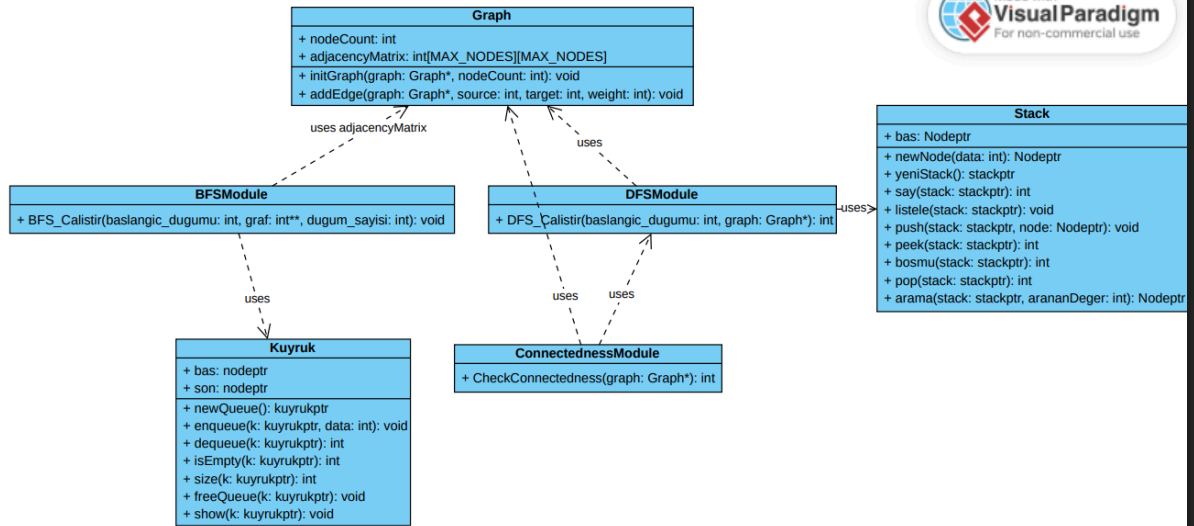
8.5 Use Case

Projenin genel çalışma akışını göstermektedir. Süreç, kullanıcının PCB graf verilerini girmesiyle başlar. Daha sonra girilen verilere göre graf yapısı oluşturulur. Graf oluşturulduktan sonra BFS veya DFS algoritması kullanılarak grafın bağlı olup olmadığı kontrol edilir.

Eğer graf bağlı değilse kullanıcıya bağlı olmayan graf uyarısı gösterilir ve işlem sonlandırılır. Eğer graf bağlıysa Kruskal algoritması ile minimum spanning tree hesaplanır. Bu sırada Union-Find veri yapısı kullanılarak döngü oluşumu engellenir.

MST kenarları seçildikten sonra toplam bağlantı maliyeti hesaplanır ve sonuç kullanıcıya gösterilir. Kullanıcı yeni düğüm veya bağlantı eklemek isterse graf güncellenir ve işlem akışı tekrar graf oluşturma adımına döner. Böylece sistem kullanıcı etkileşimine açık bir şekilde çalışır.

8.4 BFS, DFS ve Connectedness Sınıf Diyagramı



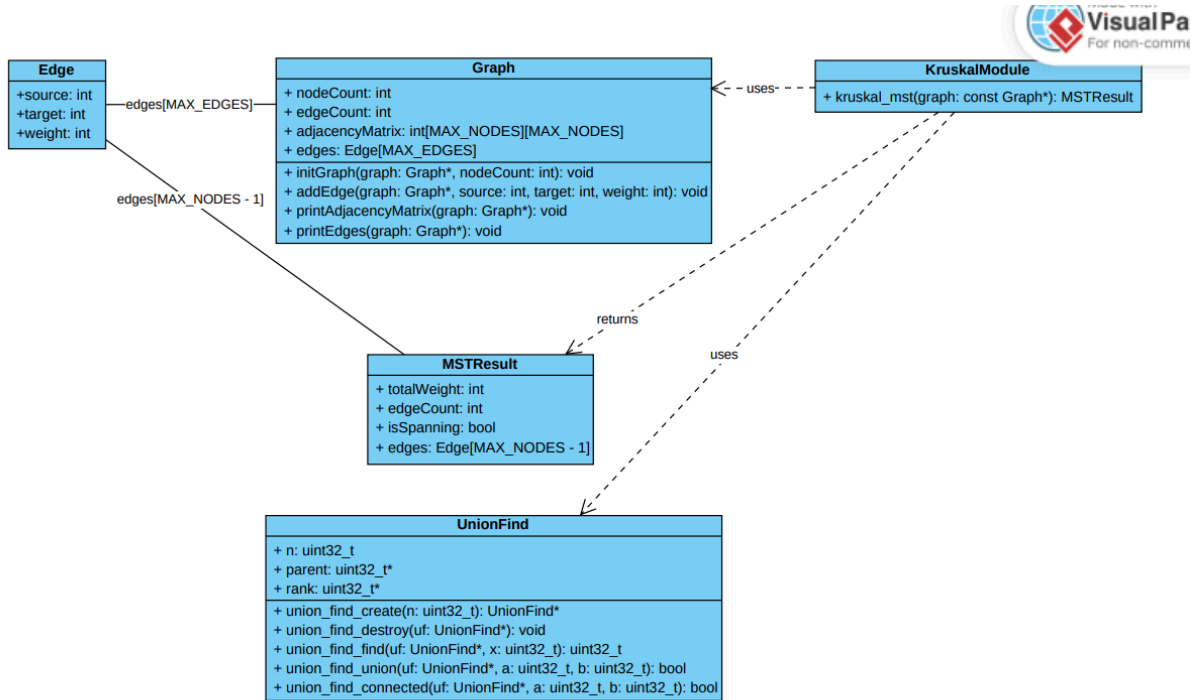
Graf üzerinde gezme ve bağlılık kontrolü yapan modülleri göstermektedir. Graph yapısı, düğüm bilgilerini ve komşuluk matrisini tutar. BFS ve DFS modülleri bu graf yapısını kullanarak graf üzerinde gezme işlemleri gerçekleştirir.

BFSModule, genişlik öncelikli arama algoritmasını çalıştırır ve bu işlemde Kuyruk veri yapısından yararlanır. Kuyruk yapısı FIFO mantığıyla çalıştığı için BFS algoritmasına uygundur. DFSModule ise derinlik öncelikli arama işlemini gerçekleştirir ve bu süreçte Stack yapısı kullanılabilir. Stack yapısı LIFO mantığıyla çalıştığı için DFS algoritmasına uygundur.

ConnectednessModule, grafın bağlı olup olmadığını kontrol etmek için BFS veya DFS mantığından yararlanır. Eğer başlangıç düğümünden tüm düğümlere ulaşılabiliriyorsa graf

bağlı kabul edilir. Bu kontrol, Kruskal algoritması çalıştırılmadan önce önemlidir çünkü bağlı olmayan bir grafta tüm düğümleri kapsayan MST oluşturulamaz.

8.4 Kruskal, Graph ve Union-Find Sınıf Diyagramı



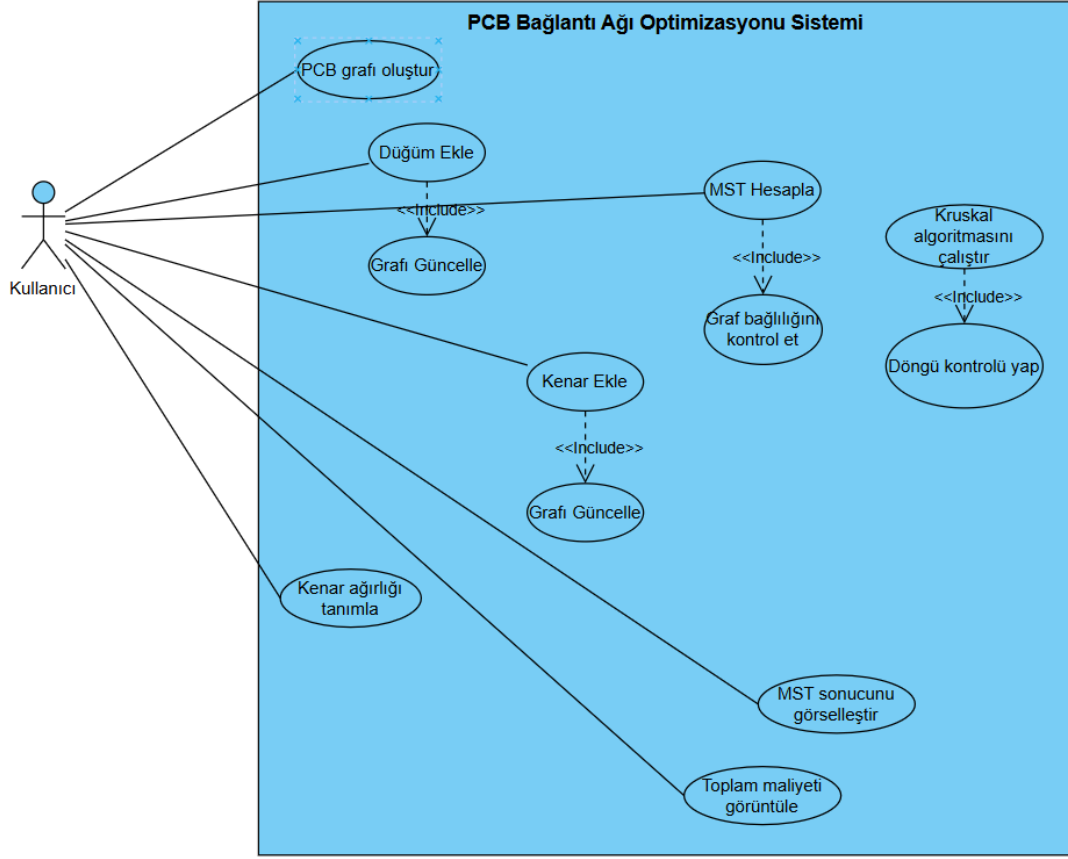
Kruskal algoritmasıyla ilişkili temel veri yapılarını göstermektedir. Graph yapısı, grafin düğüm sayısını, kenar sayısını, komşuluk matrisini ve kenar listesini tutar. Edge yapısı ise iki düğüm arasındaki bağlantının kaynak, hedef ve ağırlık bilgisini içerir.

KruskalModule, Graph yapısını kullanarak minimum spanning tree hesaplar. Algoritma, kenarları ağırlıklarına göre değerlendirir ve en düşük maliyetli bağlantıları seçer. Seçilen kenarların döngü oluşturup oluşturmadığını kontrol etmek için UnionFind veri yapısı kullanılır.

UnionFind yapısı, düğümlerin hangi kümeye ait olduğunu takip eder. İki düğüm aynı kümede ise aralarına kenar eklemek döngü oluşturacağı için bu kenar MST'ye eklenmez. Eğer iki düğüm farklı kümelerdeyse kenar MST'ye eklenir ve kümeler birleştirilir.

Sonuç olarak MSTResult yapısı oluşturulur. Bu yapı toplam bağlantı maliyetini, seçilen MST kenarlarını, kenar sayısını ve sonucun geçerli bir spanning tree olup olmadığını tutar.

8.5 Use Case



PCB bağlantı ağı optimizasyonu sisteminde kullanıcının gerçekleştirebildiği temel işlemleri göstermektedir. Kullanıcı sistem üzerinden PCB grafi oluşturabilir, düğüm ve kenar ekleyebilir, kenar ağırlıklarını tanımlayabilir ve MST hesaplama işlemini başlatabilir. Sistem, MST hesaplama sürecinde graf bağıllığını kontrol eder, Kruskal algoritmasını çalıştırır ve döngü kontrolünü gerçekleştirir. Hesaplama tamamlandıktan sonra MST sonucu görselleştirilir ve toplam bağlantı maliyeti kullanıcıya gösterilir. Ayrıca kullanıcı yeni düğüm veya kenar eklediğinde graf güncellenerek işlem süreci devam ettirilebilir.

9. JSON Tabanlı İletişim Yapısı

Frontend ve backend arasındaki iletişim JSON dosyaları üzerinden sağlanmıştır. Bu yaklaşım, iki farklı teknolojinin birbirinden bağımsız çalışabilmesini sağlar.

Kullanılan temel dosyalar şunlardır:

- input_graph.json
- output_mst.json
- calculate.flag

`input_graph.json`, kullanıcı tarafından girilen graf bilgisini içerir. Backend bu dosyayı okuyarak graf yapısını oluşturur. `calculate.flag` dosyası backend'e hesaplama yapılması gerektiğini bildirir. Backend işlemi tamamladıktan sonra sonucu `output_mst.json` dosyasına yazar.

Bu yapı sayesinde frontend ve backend arasında basit, anlaşılır ve dosya tabanlı bir iletişim kurulmuştur.

10. AI API Prompt Dokümantasyonu

Proje geliştirme sürecinde AI destekli araçlardan kod açıklama, dokümantasyon oluşturma, algoritma analizi ve rapor yazımı gibi konularda destek alınmıştır. Kullanılan promptlar, projenin geliştirme sürecini hızlandırmak ve dokümantasyon kalitesini artırmak amacıyla hazırlanmıştır.

Prompt 1: Algoritma Açıklaması

11. Test ve Doğrulama

Projenin doğruluğunu test etmek için örnek graf verileri kullanılmıştır. Kullanıcı arayüzünden girilen graf bilgileri JSON formatında backend'e aktarılmıştır. Backend tarafında grafın bağıllık durumu kontrol edilmiş ve bağlı graf üzerinde Kruskal algoritması çalıştırılmıştır.

Test sürecinde aşağıdaki durumlar kontrol edilmiştir:

- Graf verisinin JSON dosyasına doğru yazılması
- Backend'in flag dosyasını algılaması
- JSON dosyasının backend tarafından doğru okunması
- Grafın bağlı olup olmadığının kontrol edilmesi
- Kruskal algoritmasının minimum spanning tree üretmesi
- Sonucun **output_mst.json** dosyasına doğru yazılması
- Frontend'in sonucu kullanıcıya göstermesi
- Docker Compose ile sistemin tek komutla çalışması

Docker testi sonucunda frontend ve backend servislerinin başarıyla ayağa kalktığı gözlemlenmiştir. Backend servisinde sistemin başlatıldığı ve arayüzden hesaplama komutu beklediği görülmüştür. Frontend servisi ise **http://localhost:8050** adresinde çalışmıştır.

12. Karşılaşılan Problemler ve Çözümler

Geliştirme sürecinde Docker yapılandırması sırasında backend container'ının başlatılamaması problemi yaşanmıştır. Hata mesajında **./PCBNetworkOptimization** çalıştırılabilir dosyasının bulunamadığı belirtilmiştir.

İlk incelemede CMake yapılandırmasının doğru olduğu görülmüştür. **CMakeLists.txt** dosyasında proje adı **PCBNetworkOptimization** olarak tanımlanmış ve çalıştırılabilir dosya bu isimle oluşturulmuştur. Ancak **docker-compose.yml** dosyasında backend için tüm proje dizininin **/app** dizinine volume olarak bağlandığı görülmüştür.

Bu durum, build aşamasında container içinde oluşturulan çalıştırılabilir dosyanın runtime aşamasında host klasörü tarafından ezilmesine neden olmuştur. Çözüm olarak backend için tüm proje dizini yerine yalnızca frontend ile ortak kullanılan **ui/shared_data** klasörü volume olarak bağlanmıştır.

Bu düzenlemeden sonra backend ve frontend servisleri başarıyla çalışmıştır.

13. Sonuç

Proje kapsamında PCB bağlantı noktaları graf veri yapısı ile modellenmiş ve minimum maliyetli bağlantı ağını bulmak için Kruskal algoritması uygulanmıştır. Projede BFS, DFS, bağlılık kontrolü, Union-Find ve JSON tabanlı frontend-backend iletişimi gibi temel veri yapıları ve algoritmalar kullanılmıştır.

Sistem, Docker Compose ile tek komutla çalıştırılabilir hale getirilmiştir. Bu sayede kullanıcıların bağımlılık kurulumlarıyla uğraşmasına gerek kalmadan frontend ve backend servisleri birlikte başlatılabilmektedir.

Proje, veri yapıları ve algoritmalar dersinde öğrenilen graf, kuyruk, yığın, bağlılık kontrolü ve minimum spanning tree kavramlarının gerçek bir uygulama senaryosu üzerinde kullanılmasını sağlamıştır.